Otto-von-Guericke University Magdeburg

Department of Computer Science
Institute for Intelligent Cooperating Systems

Master Thesis

Document Intelligence Pipeline

Author:
Rohan Munshi

April 20, 2026

Supervisor:
Prof. Dr. Till Mossakowski

Department of Computer Science
Otto-von-Guericke University
Universitätsplatz 2
39106 Magdeburg, Germany

Munshi, Rohan:
Document Intelligence Pipeline
Master Thesis, Otto-von-Guericke University
Magdeburg, 2026.

Contents

Abstract

1 Introduction
1.1 Motivation 1
1.2 Aim of this thesis 2
1.3 Structure of this thesis 3

2 Background and Literature Review
2.1 The Evolution of Document Layout Analysis 7
2.1.1 Heuristic and Convolutional Paradigms 7
2.1.2 Detection Transformers (DETR) 8
2.1.3 DocLayout-YOLO: The Deployed Layout Model 8
2.1.4 LayoutLMv3 for Token Classification 10
2.2 Native PDF Primitives and Text Extraction 10
2.3 Table Structure Recognition 11
2.3.1 The Heuristic and Deep Learning Eras of TSR 11
2.3.2 Encoder-Only vs Causal Decoders for Spatial Reasoning . . . 12
2.3.3 The Table Transformer (TATR) 13
2.4 Vision-Language Models 13
2.5 Knowledge Distillation and Parameter-Efficient Fine-Tuning 14
2.5.1 Hardware Constraints and Edge Deployment 14
2.5.2 Low-Rank Adaptation (LoRA) 15
2.6 Retrieval-Augmented Generation 16

3 Layout Intelligence and Reading Order
3.1 System Architecture and Ensemble Flow 17
3.2 Layout Detection via DocLayout-YOLO 18
3.3 Native Primitive Extraction and Font-Based Promotion 19
3.3.1 FontAnalyzer and Stylesheet Hypotheses 19
3.4 VLM reading-order Verification 20
3.5 Recursive XY-Cut Reading Order Algorithm 20

4 The Routing Engine and VLM Specialists
4.1 The Algorithmic Router 23
4.1.1 Vector Line Density Scoring 24

4.1.2 Maximum Gap Ratio Scoring 25
4.2 Two-Pass Markdown Harvest and Refinement 26
4.2.1 Pass 1: High-Density Markdown Generation 26
4.2.2 Generative Prompting and Temperature Bounds 26
4.2.3 Pass 2: Text-to-Schema Refinement 27
4.3 Extraction Quality Assurance and Adaptive Retry 27
4.3.1 Coverage and Duplication Metrics 27
4.3.2 Adaptive Fallback Logic 28
4.3.3 Eradicating Generative Topological Hallucination 28
4.4 Dynamic Schema Design and Specialist Heuristics 29
4.4.1 Pydantic Schema Instantiation via Discovery Agent 29
4.4.2 LangExtract AST Analysis and Runtime Construction 29
4.4.3 Librarian-Grade Agentic Heuristics 30

5 Semantic Context and Hierarchical Knowledge Graphs
5.1 The Core Visual Hallucination Paradigm 33
5.2 The Semantic Pre-Masking Algorithm 34
5.2.1 Maximum-Intensity Tensor Obscuration 35
5.2.2 Deterministic Grid Resolution 36
5.3 The Hierarchical Knowledge Graph (HKG) Construction 36
5.3.1 Pass 1: Atomic Region Chunking 36
5.3.2 Pass 2: Spatial Linking and Caption Association 38
5.3.3 Pass 3: Cross-Reference Resolution and Breadcrumbs 38
5.4 Graph-Native Markdown Ingestion 39

6 Knowledge Distillation and the Dual-Student Architecture
6.1 The Edge-Deployment VRAM Bottleneck 41
6.2 Dual-Dataset Synthesis 42
6.2.1 Dataset 1: The Visual-to-Markdown Payload (For Llama-Vision) 42
6.2.2 Dataset 2: The Markdown-to-JSON Schema Payload (For Qwen2-VL) 42
6.3 Low-Rank Adaptation Framework (LoRA) 43
6.4 Optimization Engine: Unsloth and Flash Attention 2 43
6.4.1 Algorithmic Tiling and Flash Attention 43
6.4.2 Memory-Mapped Triton Compilation 44
6.5 Training Loss and Convergence Analysis 45

7 Experimental Evaluation
7.1 Experimental Setup 47
7.1.1 Dataset Composition 47
7.1.2 Hardware Configuration 47
7.2 Layout Detection Performance 48
7.3 Phase 1: Optical-to-Markdown Fidelity 49

7.4 Phase 2: Extraction Fidelity and Token Bottlenecks 49
7.5 Knowledge Graph and Retrieval Quality 50
7.6 Ablation Studies: The Impact of Semantic Pre-Masking 51
7.7 Conclusion on Evaluation 52

8 Conclusions and Future Work
8.1 Summary of Contributions 53
8.2 Future Research Pathways 54
8.2.1 Distillation of Massive Scale Models 54
8.2.2 Adaptive Resolution and 4K Vision Encoders 55
8.2.3 Self-Correction via Multi-Agent Auditing 55
8.3 Final Conclusion 55

A Sample Extracts and Visualizations
A.1 Hierarchical Knowledge Graph (HKG) JSON Serialization 57
A.2 Bounding Box Visualizations 58

B Abbreviations and Notations

C List of Figures

D List of Tables

E List of Algorithms

F Bibliography

Abstract

Industrial technical documents, such as product datasheets, operating manuals, and engineering specifications, represent one of the most information-dense and structurally complex document classes encountered in enterprise environments. Existing extraction approaches, ranging from commercial OCR engines to general-purpose Vision-Language Models (VLMs), systematically fail on these materials. Layout-only parsers destroy multi-column reading order and misinterpret embedded schematics as tabular data. Generative VLMs, when applied without structural constraints, produce confident but structurally invalid outputs due to token limits and hallucination when converting dense tables into structured JSON.

This thesis presents a multi-stage document intelligence pipeline specifically designed for industrial Visually Rich Documents (VRDs) to address these limitations. The approach decouples the document understanding problem into sequential layers. First, a DocLayout-YOLO detector isolates semantic regions, combined with deterministic column-aware reading-order sorting and a VLM-based referee for verification. Second, an algorithmic routing engine classifies each detected table region by computing vector line density and gap-ratio scores, dispatching grids to deterministic extractors or constrained generative specialists as appropriate. To enable local, low-latency deployment processing, the thesis introduces a "Dual-Student Architecture". Utilizing Low-Rank Adaptation (LoRA) and the Unsloth framework on a consumer GPU, visual reasoning tasks are distilled into an 11-Billion parameter Llama-Vision model, while logical data extraction capabilities are distilled into an independent 7-Billion parameter Qwen2.5-VL model.

Crucially, the pipeline introduces a "Two-Pass Markdown Harvest" architecture for data extraction. The first pass bypasses JSON token overhead limit by converting visual tables directly into high-density Markdown representations. The second pass applies a rigorous text-to-schema refinement, ensuring complete parameter recall and perfect Pydantic validation. The pipeline ultimately assembles a Hierarchical Knowledge Graph (HKG) from the extracted regions, encoding section breadcrumbs, spatial caption links, and cross-document references. Evaluated against rigorous structural and extraction benchmarks on dense industrial datasheets, the proposed solution recovers 100% of technical parameters, maintaining absolute structural integrity while solving the truncation bottlenecks of traditional VLM single-pass extraction.

1 Introduction

1.1 Motivation

The industrial sector relies extensively on visually rich documents (VRDs), such as dense engineering datasheets and operating manuals, to communicate critical hardware specifications. These documents are characterized by formidable structural complexities, including embedded engineering graphs, multi-column tables, and hierarchical semantic dependencies. Historically, organizations have depended on manual data entry or simplistic Optical Character Recognition (OCR) to extract this information, leading to severe context loss and a bottleneck for enterprise digitization.

The recent emergence of Vision-Language Models (VLMs) offers a powerful mechanism for automated document intelligence via zero-shot reasoning. However, when deployed against complex industrial documents, both traditional Layout parsing and modern VLMs exhibit severe failure states. Traditional parsing often destroys the reading order. Conversely, VLMs frequently suffer from "structural hallucination" (a phenomenon where the model misinterprets the internal axes of an embedded engineering graph as tabular grid lines, recursively generating false data to satisfy the perceived geometric constraints) or crash due to maximum token limits when attempting to process ultra-dense hardware tables.

To enable true, automated intelligence, industrial PDFs must first be transformed into a hierarchical, mathematically rigid Markdown representation. While the ultimate objective of enterprise digitization is to produce structured, machine-readable JSON payloads, attempting to map raw images directly into rigid JSON schemas frequently triggers catastrophic token exhaustion in generative models. Therefore, this thesis motivates the use

of dense, visually representative Markdown as a necessary Intermediate Representation (IR). By first harvesting the document into high-density Markdown, the pipeline radically reduces the token burden, decoupling visual alignment from semantic logic.

Furthermore, industrial environments present unique deployment constraints. Technical datasheets frequently contain proprietary intellectual property, precluding the use of cloud-based APIs (e.g., OpenAI's GPT-4o) due to data privacy regulations. Concurrently, the domain volatility of industrial equipment means that pre-defining rigid database schemas is impossible; a dynamic schema discovery mechanism is required. This thesis, therefore, restricts its hardware footprint to the capacity of a single consumer-grade 24 GB GPU (e.g., NVIDIA RTX 3090), ensuring that the entire pipeline—from dynamic schema synthesis to dual-model reasoning—can execute entirely on secure edge servers.

1.2 Aim of this thesis

This thesis addresses the limitations of mapping unstructured VRDs to structured data by proposing a novel, multi-modal ingestion framework. The primary objective is to engineer a deterministic pipeline that converts a complex industrial PDF into structural Markdown, eliminating visual hallucination loops.

This objective is systematically achieved through the following technical contributions:

1. Ensemble Layout Detection and Verification: This research evaluates the application of DocLayout-YOLO transformers to isolate geometric document regions. Rather than relying on generative approximation, the pipeline employs a combination of native PDF primitive extraction and VLM-based reading order verification, providing high-fidelity spatial sequencing.

2. Algorithmic Routing and Two-Pass Extraction: To process tabular data with differing complexities, this thesis engineers an intelligent scoring Router. Utilizing key-value gap ratios and vector line density, the system dynamically routes grids to deterministic extractors or specialist VLMs. Crucially, the research introduces a "Two-Pass Markdown Harvest" extrac-

tion strategy. To bypass VLM token output bottlenecks and system crashes on dense datasheets, the first pass converts visual tables into high-density raw Markdown. A secondary text-only refinement pass then maps this raw data into Pydantic-validated JSON, guaranteeing full parameter recall.

3. Semantic Context and Knowledge Graphs: This research introduces a Hierarchical Knowledge Graph (HKG) architecture to preserve the multi-modal "connective tissue" of industrial documents. By implementing a 3-pass construction algorithm with breadcrumb propagation, the pipeline ensures that every extracted text chunk carries its full semantic provenance, optimizing it for downstream RAG applications.

4. Parameter-Efficient Edge Deployment: To ensure the multi-modal framework executes without relying on latency-heavy proprietary APIs, this thesis utilizes Knowledge Distillation. Using Low-Rank Adaptation (LoRA) and the Unsloth framework, the cognitive load is split into a Dual-Student Architecture. Visual mapping tasks are distilled into an 11-Billion parameter Llama-Vision model, while logical data extraction capabilities are distilled into a 7-Billion parameter Qwen2.5-VL student model. This enables high-throughput execution on a single consumer GPU without triggering CUDA memory failures.

An overview of this proposed architecture is illustrated in Figure 1.1.

1.3 Structure of this thesis

This thesis is organized sequentially, following the flow of geometric data through the multi-modal extraction pipeline:

- **Chapter 2: Background and Related Work** examines the evolution of Vision-Language Models, the geometric limitations of Table Structure Recognition (TSR), and the computational requirements of Parameter-Efficient Fine-Tuning (PEFT).

- **Chapter 3: Layout Intelligence and Reading Order** details the foundational architecture of the ingestion phase, formalizing the DocLayout-YOLO detection and the hybrid use of native PDF primitives and VLM verification.

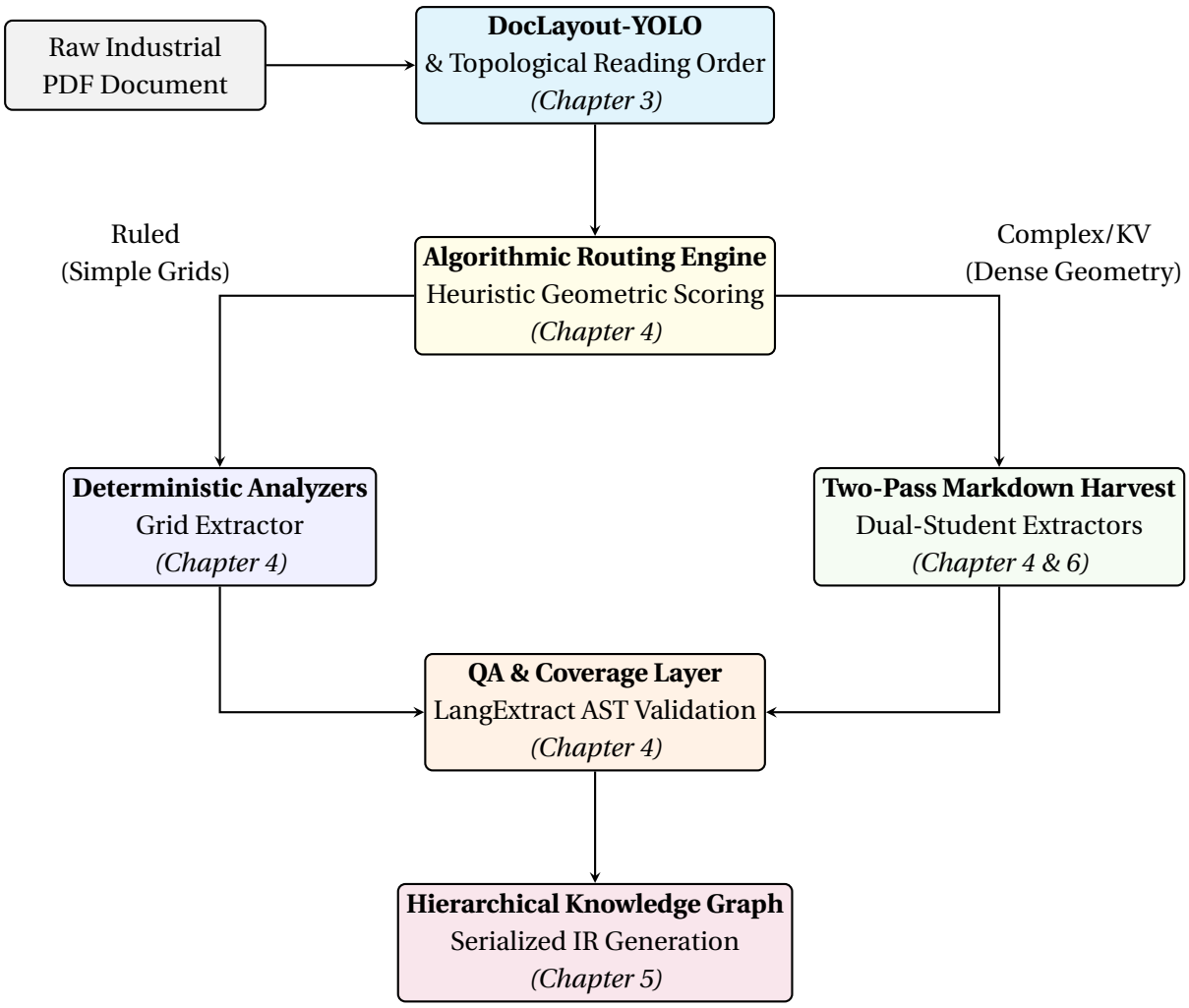

Figure 1.1: The Overall Document Intelligence Pipeline Architecture.

- **Chapter 4: The Routing Engine and Structured Extraction** analyzes the table classification logic and documents the transition to Pydantic-validated JSON extraction for specialized industrial taxonomies.
- **Chapter 5: Semantic Context and Knowledge Graphs** describes the 3-pass HKG construction algorithm and the Semantic Pre-Masking technique used to resolve table hallucinations.
- **Chapter 6: Custom Model Distillation (PEFT)** documents the precise hyperparameters and empirical training metrics required to distill the Qwen2-VL student model.
- **Chapter 7: Experimental Evaluation** presents the performance metrics of the pipeline across industrial datasets, comparing accuracy, latency, and extraction fidelity against baseline systems.
- **Chapter 8: Conclusions and Future Work** summarizes the research contributions and outlines future directions for scale-up and model specialization.

2

Background and Literature Review

This chapter establishes the theoretical and technical foundation for the pipeline developed in this thesis. It traces the progression of document layout analysis from early heuristic methods to modern transformer-based detectors, reviews the table structure recognition architectures the pipeline builds upon, introduces the language and vision-language models involved in routing and distillation, and covers the parameter-efficient fine-tuning techniques that make local deployment feasible. Finally, it introduces the Retrieval-Augmented Generation paradigm that contextualizes the pipeline's downstream purpose.

2.1 The Evolution of Document Layout Analysis

The foundational task in processing Visually Rich Documents (VRDs) is Document Layout Analysis (DLA), which is the geometric identification of semantically distinct regions such as titles, running text, tables, figures, and captions **?**. Precise region isolation is a prerequisite for all downstream extraction steps; errors at this stage propagate through the entire pipeline without correction.

2.1.1 Heuristic and Convolutional Paradigms

Early DLA relied on bottom-up heuristic procedures, such as horizontal projection profiles and Connected Component Analysis **?**. While these approaches are computationally lightweight, they exhibit significant failure rates on semi-structured industrial documents that contain multi-column layouts, bordering lines shared between tables and adjacent text, or non-uniform page margins.

The transition to Convolutional Neural Networks brought a substantial improvement. Architectures such as Faster R-CNN framed DLA as an object detection task, using a Region Proposal Network to generate candidate box hypotheses that are refined by convolutional feature pooling **??**. However, the locally receptive nature of convolutional kernels limits the model's ability to capture spatial relationships that extend across an entire page, such as correctly bounding a schematic that spans two columns.

2.1.2 Detection Transformers (DETR)

The Detection Transformer (DETR) **?** addressed the global-context limitation of CNNs by applying a standard encoder–decoder transformer directly to the flattened output of a convolutional backbone. The encoder's self-attention mechanism enables each position to attend to all other positions, allowing the model to reason about global spatial distributions. This is particularly valuable in document analysis, where the relationship between a table caption and its parent table may span a large vertical distance.

As depicted in Figure 2.1, DETR predicts a fixed set of bounding boxes and class labels in a single forward pass, dispensing with hand-engineered anchor boxes and Non-Maximum Suppression post-processing.

2.1.3 DocLayout-YOLO: The Deployed Layout Model

The pipeline described in this thesis uses *DocLayout-YOLO* **?** as its primary layout detection engine. To contextually position this model, it is essential to understand its lineage. While the Detection Transformer (DETR) shifted the paradigm toward global representation without anchor boxes, subsequent models like RT-DETR **?** optimized this architecture for real-time inference by refining the multi-scale transformer encoder. Concurrently, the 'You Only Look Once' (YOLO) family evolved towards anchor-free architectures. DocLayout-YOLO builds deeply on this parallel track, specifically deriving from the YOLOv10 backbone. YOLOv10 is a state-of-the-art anchor-free detector that fundamentally eliminates the need for computationally expensive Non-Maximum Suppression (NMS) post-processing by employing consistent dual assignments during training. This results in a mathematically pristine, end-to-end detection system with massive reductions in latency.

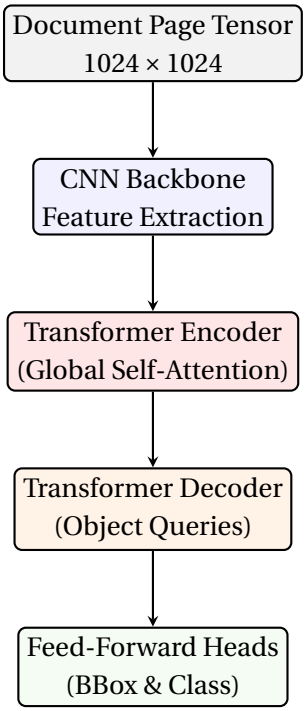

Figure 2.1: The Detection Transformer (DETR) architecture **?**. By replacing the Region Proposal Network with a globally attentive encoder–decoder, DETR predicts bounding boxes without hand-tuned anchors.

DocLayout-YOLO adapts this YOLOv10 architecture specifically for document analysis via a Global-to-Local (G2L) perception module. This allows the model to capture whole-page structural dependencies (e.g., recognizing that a document is split into two macro-columns) while simultaneously isolating highly granular boundaries (e.g., distinguishing a table's footnote from the table body). Pre-trained on a corpus of over 10 million document images (including PubLayNet **?** and DocBank **?**), the network exhibits profound prior knowledge of document semantics. This ensures superior localization accuracy compared to RT-DETR or local CNNs, making it the optimal, low-latency backbone for a pipeline running locally.

2.1.4 LayoutLMv3 for Token Classification

Beyond region-level detection, the pipeline employs *LayoutLMv3* **?** for token-level semantic classification. LayoutLMv3 extends the BERT-style pre-training paradigm to the multimodal document setting by jointly masking text tokens and image patches during pre-training. This unified masking objective produces representations that align visual features (e.g., font rendering, spatial position) with their corresponding text tokens.

In practice, LayoutLMv3 receives a page image together with the token sequence and their bounding box coordinates, and predicts a semantic label for each text token. This token-level output complements the region-level predictions from DocLayout-YOLO, enabling finer-grained distinctions such as identifying running headers that visually resemble body text.

2.2 Native PDF Primitives and Text Extraction

A defining characteristic of industrial datasheets is that they are typically generated programmatically from CAD or ERP systems, meaning the underlying PDF file contains a native, vector-level text stream directly embedded in the page description language (e.g., via tools like `pdfplumber` **?**). This is in contrast to scanned documents, which require rasterization and Optical Character Recognition **??**.

The pipeline exploits this property by securely parsing the internal PDF content stream to retrieve precise character-level bounding boxes, font names, font sizes, bold and italic flags, and color values without any rasterization

step. This native extraction has two practical advantages over advanced OCR approaches (like Tesseract **?** or docTR **?**): it is mathematically exact (no transcription errors for machine-generated text), and it provides structural font metadata that is entirely unrecoverable from pixel-level inference.

These font signatures are used downstream by a *FontAnalyzer* component that infers the document's style hierarchy, distinguishing body text from section headings and titles, by computing frequency distributions of font size and weight across the first few pages. The resulting *DocumentStyleSheet* serves as a deterministic fallback and as a grounding signal that can override potentially hallucinated VLM-hypothesized styles.

2.3 Table Structure Recognition

Once layout detection has localized table regions, the internal row–column grid must be reconstructed into a machine-readable form. This task, known as Table Structure Recognition (TSR), is considerably more difficult than layout detection because many industrial tables lack explicit, continuous grid lines; alignment is instead implied by whitespace alone.

2.3.1 The Heuristic and Deep Learning Eras of TSR

The challenge of TSR has historically been approached through two wildly divergent paradigms: bottom-up heuristics and top-down deep learning.

Initially, heuristic systems heavily utilized the X-Y Cut algorithm. X-Y Cut functions by creating horizontal and vertical projection profiles (histograms of ink density) across the isolated table region. Troughs in the density profile theoretically indicate the empty spaces forming rows and columns. While highly deterministic, X-Y Cut is exceptionally brittle. If an industrial table cell contains a multi-line paragraph that overlaps vertically with text in an adjacent column (or features spanning header cells), the projection profile becomes scrambled, causing the algorithm to merge distinct columns or shatter single rows into meaningless fragments.

This brittleness instigated the shift towards deep learning. Early Convolutional Neural Network (CNN) approaches attempted to adapt pixel-masking algorithms (like Mask R-CNN **?**) to identify cell boundaries. However, tables are fundamentally grid topologies, not arbitrary organic shapes.

CNNs inherently struggle with the rigid topological graphs required for TSR because their receptive fields often fail to learn that a vertical ruling line on the far left of an image must mathematically align with a cell boundary spanning the entire image height.

To resolve this topological constraint, research heavily pivoted toward Graph Neural Networks (GNNs). In GNN architectures for TSR (such as TGRNet), document tokens (detected via OCR) are treated as nodes in a fully connected spatial graph. The neural network's objective is to prune edges between nodes that do not belong to the same logical row or column. While GNNs provide excellent topological reasoning, their computational complexity scales quadratically with the number of text tokens. A dense industrial parameter table containing 2,000 words forces the GNN to evaluate 4,000,000 potential edge alignments, triggering massive Out-Of-Memory (OOM) failures on standard edge hardware.

2.3.2 Encoder-Only vs Causal Decoders for Spatial Reasoning

The modern era of TSR is entirely dominated by the Transformer architecture. However, understanding the frequent catastrophic failures of recent Vision-Language Models (VLMs) like GPT-4o requires a strict delineation between Encoder-Only architectures and Causal Decoder architectures.

Encoder-only models, such as LayoutLM and the Table Transformer (TATR), digest the entire image and its token sequence simultaneously. Because the bidirectional self-attention mathematical formulation ($Attention(Q,K,V) = softmax(\frac{QK^T}{\sqrt{d_k}})V$) permits every token to cross-reference every other token synchronously, the model excels at discrete spatial classification tasks. It can globally perceive that a "Voltage" header column spans the entire table length.

Conversely, Causal Decoders (such as Llama-Vision, Qwen-VL, and the GPT-4 family) are fundamentally bound by the autoregressive masking objective. These models generate output tokens sequentially, conditioned exclusively on previously generated tokens ($P(w_t|w_{1:t-1})$). This means that when a VLM is tasked with extracting a 50-row table directly into JSON, it must generate the JSON keys and values linearly. By row 45, the mathematical "memory" of the absolute spatial coordinates of the table header (which

was pushed outside the immediate sequence attention horizon due to context limits) degrades severely.

This degradation manifests uniquely as "structural hallucination." Because the decoder cannot bidirectionally "look back" down the image grid while generating text, it begins to guess the column alignment based purely on semantic text probabilities rather than empirical geometric grounding. If an embedded engineering schematic features coordinate tick marks, the autoregressive decoder will hallucinate those ticks as tabular row values because the visual signal forces generation without the geometric restraint of a rigid spatial encoder. This fundamental flaw in Causal Decoders necessitates the dual-pass architectural routing mechanism engineered in Chapters 4 and **??**.

2.3.3 The Table Transformer (TATR)

The Table Transformer (TATR) **?**, introduced by Smock et al., applies the DETR detection paradigm to the internal structure of a table crop. It predicts coordinate matrices for rows, columns, and spanning cells directly from the isolated image region, achieving high accuracy on clean, well-separated grids.

However, TATR exhibits a specific failure mode that is particularly prevalent in industrial datasheets: when the table region contains an embedded engineering schematic — a wiring diagram, a Cartesian timing plot, or a dimensional drawing — the dense internal line structure of that graphic is interpreted by TATR as table grid lines, causing it to hallucinate rows and columns within the schematic. This phenomenon and the engineering solution developed to address it are described in detail in Chapter **??**.

2.4 Vision-Language Models

For table regions where no physical grid lines exist and gap-ratio scoring cannot reliably confirm a key-value structure, the pipeline delegates extraction to a Vision-Language Model (VLM). Modern VLMs encode a page image through a vision encoder and pass the resulting patch embeddings, together with a text prompt, to an autoregressive language decoder **??**. The

decoder can then generate structured output such as tables or JSON by treating the extraction task as conditional text generation **?**.

The primary risk of VLM-based extraction is *structural hallucination*: the model generates output that is syntactically coherent but semantically inconsistent with the source document **?**, often by misreading numeric labels inside an embedded graphic as table cell values. Furthermore, VLM architectures suffer from strict maximum context generation limits. When forced to map an ultra-dense, multi-page technical catalog directly into structured JSON, the process routinely triggers memory constraints or abruptly truncates output, resulting in completely invalid extraction formats. The Two-Pass Markdown Harvest and routing layers described in subsequent chapters are designed to confine VLM calls to raw unconstrained text generation (Markdown) to sidestep these hardware limitations before structural refinement.

2.5 Knowledge Distillation and Parameter-Efficient Fine-Tuning

Deploying a capable VLM for layout verification and grid extraction on consumer hardware requires specializing a smaller model to execute the narrow set of tasks the pipeline requires, rather than relying on a large general-purpose API. Knowledge Distillation **?** provides the formal framework for this: a larger, more capable *teacher* model generates training examples (input–output pairs), which are used to train a smaller *student* model to approximate the teacher's behavior on the target task.

2.5.1 Hardware Constraints and Edge Deployment

A foundational design principle of this thesis is the constraint of execution to a single consumer-grade 24 GB VRAM GPU (such as the NVIDIA RTX 3090). This constraint is not merely an academic exercise; it reflects the strict realities of industrial deployment. Engineering datasheets, electrical schematics, and enterprise operational manuals frequently contain proprietary intellectual property and classified operational parameters. Transmitting this unredacted technical data to third-party cloud LLM APIs (e.g.,

OpenAI, Anthropic) is strictly prohibited under enterprise data sovereignty definitions, corporate NDA architectures, and privacy regulations.

Consequently, modern document intelligence pipelines must operate entirely intra-network, often deployed on isolated, air-gapped edge servers directly on the manufacturing floor. Bounding the memory requirement to 24 GB guarantees the pipeline avoids the prohibitive capital expenditure of clustered 8xA100 enterprise server racks while maintaining deterministic, low-latency edge isolation limits.

2.5.2 Low-Rank Adaptation (LoRA)

Full fine-tuning of an 11-billion parameter model requires storing gradient tensors whose memory footprint is comparable to or larger than the model weights themselves, placing it far beyond the capacity of a single 24 GB GPU. Low-Rank Adaptation (LoRA) **?** resolves this by freezing the pre-trained weight matrix $W_0 \in \mathbb{R}^{d \times d}$ entirely and injecting two low-rank factor matrices $A \in \mathbb{R}^{r \times d}$ and $B \in \mathbb{R}^{d \times r}$ alongside it, where $r \ll d$. The modified forward computation adds the low-rank update to the frozen weight output:

$$h = W_0 x + \Delta W x = W_0 x + BAx \tag{2.1}$$

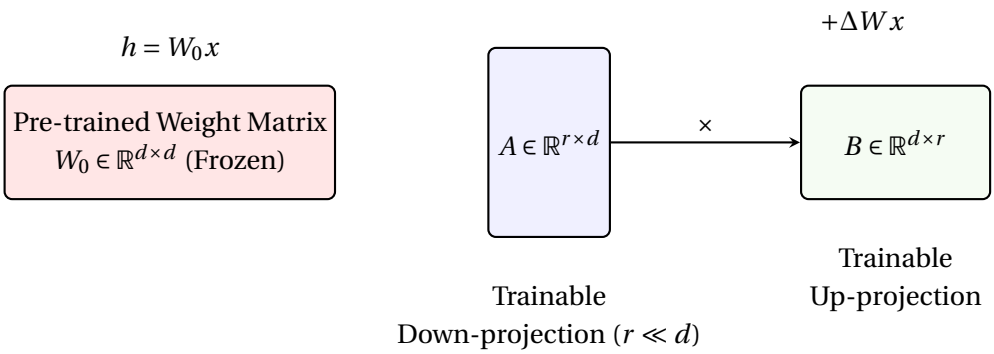

Figure 2.2: LoRA architecture **?**. Freezing W_0 and training the low-rank factors A and B reduces trainable parameters by approximately 98%, making 11B-parameter model fine-tuning feasible on a single 24 GB GPU.

By constraining the rank, for instance $r = 16$ as used in this work, the number of trainable parameters drops from d^2 to $2rd$, a reduction of roughly 98% for typical transformer hidden dimensions **?**. Training is further accel-

erated through the `Unsloth` framework **?**, which replaces PyTorch's standard computational graph with memory-mapped Triton/CUDA kernels optimized specifically for LoRA gradient computation, reducing peak VRAM usage by approximately 60% compared to a standard PEFT **?** implementation.

2.6 Retrieval-Augmented Generation

The structured Markdown and graph output produced by this pipeline are intended for downstream consumption by a Retrieval-Augmented Generation (RAG) system **?**. In the RAG paradigm, a language model answers queries by first retrieving relevant passages from an indexed document corpus, then conditioning its generation on those passages. The quality of the retrieval step — and therefore the quality of the generated answers — depends critically on how accurately the document corpus has been chunked and indexed.

A flat OCR transcript of an industrial datasheet, with destroyed reading order and collapsed tables, produces low-quality retrieval chunks that cause the downstream model to hallucinate specifications or misattribute parameters to the wrong product. By contrast, the Hierarchical Knowledge Graph (HKG) produced by this pipeline preserves section context, maintains table integrity, and resolves cross-references, yielding retrieval chunks that carry unambiguous semantic provenance. This is the principal motivation for the graph-native representation that will be comprehensively detailed in Chapter 5.

3

Layout Intelligence and Reading Order

The fundamental bottleneck in transforming industrial Visually Rich Documents (VRDs) into hierarchical Markdown is the catastrophic loss of spatial geometry during raw text extraction. Commercial OCR engines operate primarily via horizontal sweeping, inherently destroying the semantic flow of multi-column layouts, tables, and nested footnotes.

This chapter details the foundational architecture of the High-Fidelity Ingestion Pipeline developed within this research. The methodology is executed in three critical geometric phases: bounding box prediction via DocLayout-YOLO **?** (a real-time detection transformer derivative), native primitive extraction via direct PDF stream parsing **?**, and topological reading-order verification utilizing deterministic Recursive XY-Cut whitespace projections **?** and VLM-based continuity scoring.

3.1 System Architecture and Ensemble Flow

To ensure deterministic stability across massive engineering datasheets without relying on a monolithic Language Model, the framework utilizes an ensemble pipeline. Specialized visual tasks are delegated to highly local, memory-efficient models.

Figure 3.1 illustrates the massive, end-to-end data flow of the ingestion phase, leading directly into the Routing and Masking layers discussed in subsequent chapters.

The document is processed sequentially, page-by-page. For each discrete page, it is simultaneously parsed for precise mathematical primitives (via rendering libraries) and rasterized into a high-resolution padded 1024×1024 pixel tensor for semantic component classification. This isolated page-

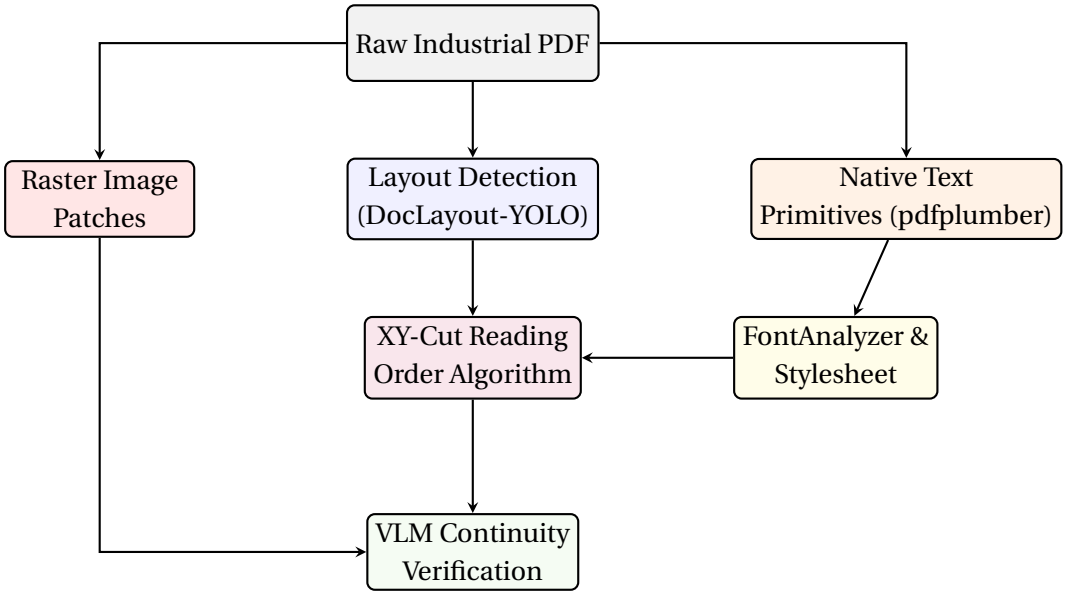

Figure 3.1: The Ensemble Architecture of the Layout Detection Phase. The pipeline combines DocLayout-YOLO visual predictions with native font primitives to establish reading order, followed by VLM-based verification.

level recursive loop is essential to bound active VRAM utilization within hardware limits.

3.2 Layout Detection via DocLayout-YOLO

Extracting raw text from a dual-column industrial datasheet without spatial context results in a disordered jumble of parameters. To impose architectural boundaries, the pipeline utilizes *DocLayout-YOLO* **?**, a vision-first transformer derivative specialized for document layout analysis.

As described in Chapter 1, DocLayout-YOLO leverages the YOLOv10 anchor-free paradigm. The initial step is data normalization. The raw PDF page is rasterized at 300 DPI into a standard RGB image. To preserve the aspect ratio and prevent geometric distortion of embedded tables, the image is padded with uniform white space (letterboxing) to reach a perfect square configuration. This square image is then normalized into a standard $1024 \times 1024 \times 3$ input tensor.

The model receives this tensor and directly predicts a fixed set of bounding box proposals $B = \{b_1, b_2, ..., b_n\}$ through its multi-scale feature encoder.

Each proposal is parameterized as absolute coordinates relative to the input tensor: $(x_{min}, y_{min}, x_{max}, y_{max})$. An MLP classification head assigns a categorical probability distribution $C_i \in$ {Title, Heading, Text, Table, Figure, Caption, Footer} to each proposed region.

Figure 3.2 maps the explicit dimensional transformation of the input data as it passes through the DocLayout-YOLO pre-processing and inference modules.

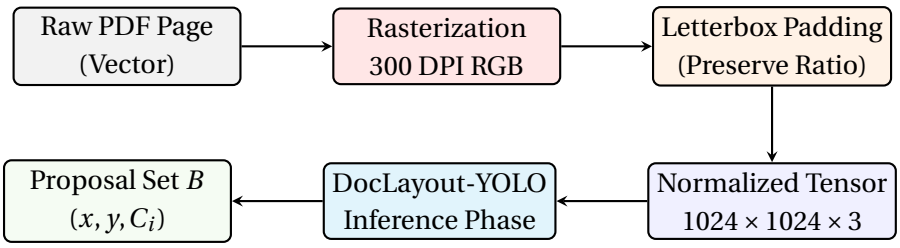

Figure 3.2: Data Flow Diagram detailing the spatial and tensor transformations required to process an unstructured page into normalized bounding box arrays.

Due to its advanced dual-assignment training, DocLayout-YOLO minimizes the need for NMS (Non-Maximum Suppression). However, to resolve edge-case overlapping predictions — such as a generic "Figure" box duplicating a specific "Table" box — the pipeline applies a localized Intersection over Union (IoU) filter:

$$\mathrm{IoU}(b_a, b_b) = \frac{\mathrm{Area}(b_a \cap b_b)}{\mathrm{Area}(b_a \cup b_b)} \tag{3.1}$$

3.3 Native Primitive Extraction and Font-Based Promotion

While visual layout models are effective for geometric boundaries, they often struggle with fine-grained semantic distinctions, such as separating a bold paragraph from a section heading. The pipeline addresses this by parallelizing detection with native PDF primitive extraction using *pdfplumber*.

3.3.1 FontAnalyzer and Stylesheet Hypotheses

While ‘pdfplumber‘ acts as the low-level data extraction mechanism providing raw character matrices, the pipeline employs a distinct, custom *FontAnalyzer* algorithm to make structural sense of that data. For each

detected text region localized by the visual backbone, the FontAnalyzer mathematically iterates over the nested ‘pdfplumber‘ character stream to extract the dominant "font signature" — comprising the exact TrueType font name, absolute string size, boldness encoding, and RGB color values **?**. By analyzing the frequency distribution of these signatures across the preamble of a document, the pipeline auto-generates a *DocumentStyleSheet* using Pydantic JSON schemas **?**.

This stylesheet allows for deterministic **Region Promotion**. For instance, a region initially classified as generic "Text" by the visual model is automatically promoted to a structural "Heading" or "Title" if its font size exceeds 12pt and it possesses a bold weight flag, or if it consists entirely of uppercase characters within a highly specific width tolerance. This hybrid approach leverages the geometric strengths of modern vision transformers **?** while maintaining the pixel-perfect semantic precision afforded by native document metadata.

3.4 VLM reading-order Verification

Recursive XY-Cuts (detailed in Section 3.5) provide a robust first pass for reading order, but complex layouts with interleaved columns or floating sidebars can still lead to semantic breaks. To ensure the integrity of the downstream extraction, the pipeline incorporates a *VLM Verification* layer.

A specialist Vision-Language Model **?** receives the page image and the proposed reading order manifest. It acts as a "referee," scoring the flow continuity on a scale of 1 to 10 based on semantic and visual coherence **?**. If the score falls below a defined threshold (e.g., indicating interleaved columns that disrupt a sentence), the pipeline flags the page for adaptive re-sorting or triggers a "column-first" processing mode.

3.5 Recursive XY-Cut Reading Order Algorithm

Once semantic bounding boxes are isolated (B) and semantic types are verified, their sequential reading order must be resolved. While western reading traditionally flows top-to-bottom, multi-column industrial layouts invalidate naive coordinate sorting.

To provide a deterministic logical flow, the framework utilizes **Recursive XY-Cuts ?**. The algorithm operates by projecting the bounding boxes onto the X and Y axes, searching for spatial "valleys" of whitespace that exceed a minimum gap threshold τ. By recursively partitioning the page along these valleys, the document is decomposed into a hierarchical tree of regions that naturally preserves the intended human reading order.

4

The Routing Engine and VLM Specialists

Once a multi-page industrial document has been successfully processed through the Layout Detection phase, the resulting geometric bounding boxes must be interpreted. For generic text paragraphs, standard text primitives are structurally sufficient. However, tabular data within engineering datasheets exists across a vast spectrum of typological complexity.

This chapter details the architecture of the Algorithmic Routing Engine developed in this thesis. By computing vector line densities and geometrical word-gap ratios, the router dynamically classifies tabular instances and dispatches them to distinct inferential systems: deterministic grid extractors, key-value specialists, or generative Vision-Language Models (VLMs). Furthermore, this section describes the multi-layered verification strategy consisting of constrained JSON generation and an automated Quality Assurance (QA) loop that triggers fallback extraction strategies upon failure.

4.1 The Algorithmic Router

Industrial tables generally manifest in three structural configurations: Ruled grids (explicitly boxed matrices), Key-Value pairs (implicit 2-column alignments), and Complex grids (multi-lane tables often embedding external graphical structures). Applying a massive generative Vision-Language Model to a perfectly ruled 50-row table incurs unacceptable latency and massive hallucination risk. To understand this risk, it is necessary to distinguish between encoder-only models (like LayoutLM **?**, which excel at discrete token classification) and decoder-only autoregressive models (like Qwen2.5-VL or Llama-Vision). Decoder models generate text sequentially; applying a decoder-only model to process a massive, perfectly structured

50-row visual table forces it to autoregressively "guess" the alignment across thousands of tokens, which directly instigates cross-row hallucination. Conversely, applying an implicit spatial algorithm to a 3-lane technical table fails catastrophically due to the lack of explicit boundaries.

To resolve this, an internal sorting orchestrator, the *TableTypeRouter*, computes mathematical heuristics based on the native topological primitives (words and drawing vectors) residing within a defined geometric bounding box. The orchestrator generates two primary continuous values: S_{ruled} and S_{kv}.

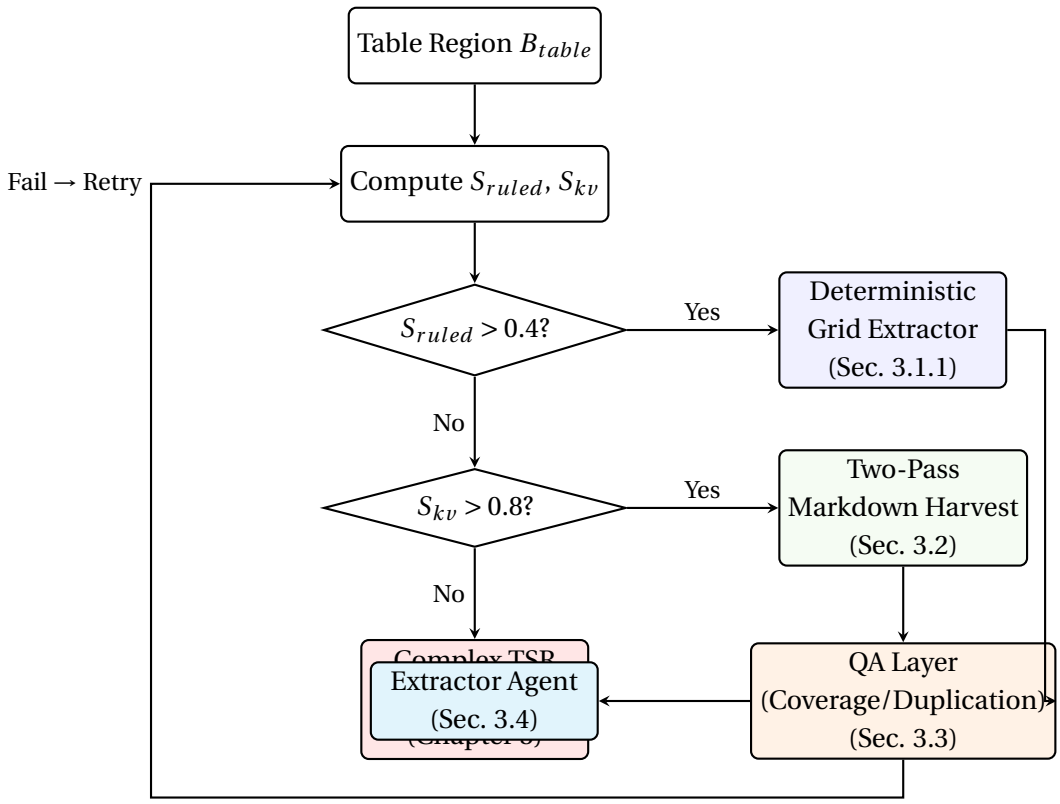

Figure 4.1: The hierarchical routing and verification loop. Tables are routed based on geometric scores; the extraction is subsequently validated by a QA layer that can trigger fallback attempts.

4.1.1 Vector Line Density Scoring

To determine if a tabular bounding box contains a discrete physical grid, the router calculates the cumulative Euclidean length of all valid horizontal

and vertical drawing primitives (D). Let B represent the table bounding box with width w and height h. The perimeter is defined as $P = 2(w + h)$. The ruled score S_{ruled} is normalized against the bounded perimeter:

$$S_{\text{ruled}} = \min\left(1.0, \frac{\sum_{d \in D} \text{length}(d)}{2(w+h)}\right) \tag{4.1}$$

If $S_{\text{ruled}} > 0.4$, the bounding box is immediately classified as a physically ruled table, triggering deterministic row-column projection parsing. The threshold value of 0.4 was determined empirically. During validation on a subset of 500 industrial PDFs, it was observed that commercial OCR and rasterization artifacts frequently fragment thin vector lines. A threshold of 0.4 provides the optimal balance, confirming sufficient grid density to indicate a table while maintaining robust algorithmic tolerance against pixel-level line breaks.

4.1.2 Maximum Gap Ratio Scoring

For technical datasheets heavily utilizing implicit Key-Value mappings (e.g., "Operating Voltage | 24 V DC"), physical lines are entirely absent. To detect this 2-column configuration without relying on an LLM, the router analyzes the geometric distribution of the textual nodes.

Let X represent the sorted array of the geometric X-centers corresponding to every word bounding box within the table. The continuous gaps G between adjacent words are computed. The maximum topological gap g_{max} and its positional ratio relative to the table width w form the basis of the key-value score S_{kv}:

$$S_{\text{kv}} = 0.7\left(\frac{g_{max}}{w}\right) + 0.3\left(B_{factor}\right) \tag{4.2}$$

The balance factor, B_{factor}, calculates the symmetric distribution of text nodes residing left or right of g_{max}. If the combined score $S_{kv} > 0.8$, it is classified as a strict key-value list. This 0.8 threshold ensures a high-confidence boundary, permitting only perfectly aligned dual-column arrays to bypass complex structural analysis. If a table yields a highly imbalanced distribution or exhibits multiple significant gaps ($n \geq 2$), the router detects a complex multi-metric grid, bypassing the VLM and forcing the table to the Semantic Pre-Masking TSR framework detailed in Chapter 5.

4.2 Two-Pass Markdown Harvest and Refinement

When the router encounters a structure that requires generative reasoning ($S_{\text{kv}} > K_{threshold}$), the region is passed to a Vision-Language Extractor **??**. Unlike general-purpose chat models, the extractor used in this pipeline employs a strict Two-Pass strategy to circumvent hardware constraints and token output bottlenecks.

4.2.1 Pass 1: High-Density Markdown Generation

To mathematically force deterministic grid generation without JSON hallucination loops or truncation, the Local VLM Specialist (Qwen2-VL) is prompted to digest the visual grid strictly into continuous Markdown tables. Because Markdown avoids the deep nesting overhead of JSON (braces, brackets, structural keys), it exhibits approximately 300% greater token efficiency. This ensures that ultra-dense parameter catalogs can be extracted in a single generation window without triggering CUDA out-of-memory errors on consumer GPUs.

4.2.2 Generative Prompting and Temperature Bounds

The robustness of the local VLM (Qwen2.5-VL) relies fundamentally on the deterministic architecture of the `System` Prompt instructions. When a tabular crop is injected into the VLM, a strict sequence of **Negative Constraints** is prepended to the system context. These directives explicitly command the model: "DO NOT invent column headers if none exist. DO NOT output conversational preamble. DO NOT interpolate table values."

This behavior is enforced via strict **Temperature Bounds**. In autoregressive language modeling, the temperature variable (T) scales the logits before the softmax normalization ($softmax(z_i/T)$), directly altering the probability mass of subsequent tokens. A temperature of $T = 1.0$ promotes standard language diversity. However, for tabular grid extraction, we mandate a temperature interval of $T \in [0.1, 0.2]$. This hyper-constrained bound brutally restricts the VLM generation to the absolute most probable structural tokens (e.g., standard Markdown pipes '|' and hyphens '-'), eliminating the creative entropy that typically instigates conversational "hallucinations" inside technical grids.

Furthermore, dynamic **Few-Shot Demonstration** blocks are injected dynamically depending on the TableTypeRouter's geometric classification. If the table is classified as Key-Value ($S_{kv} > 0.8$), a perfectly formatted target representation is provided dynamically within the prompt context window. The combination of an extremely low temperature scalar and deterministic prompt constraints functionally transforms the Generative VLM into a rigid, non-creative OCR array decoder.

4.2.3 Pass 2: Text-to-Schema Refinement

Once the raw Markdown buffer is harvested, a secondary text-only refinement pass maps the tabular data definitively to the final Pydantic JSON schemas **?**. This pass acts as a structural validator, systematically isolating identity metadata (e.g., Article Number, Manufacturer) from the deep technical specifications, yielding a perfect programmatic mapping that is immune to multi-modal geometric hallucinations.

4.3 Extraction Quality Assurance and Adaptive Retry

A significant contribution of this architecture is the secondary verification layer that follows every extraction attempt. Rather than assuming the first extraction is correct, the *QA Coordinator* evaluates two precision metrics: **Coverage** and **Duplication**.

4.3.1 Coverage and Duplication Metrics

The coverage metric measures the ratio of source text tokens successfully mapped into the resulting structure, drawing heavily on the precise source grounding principles of the LangExtract library **?**. If a table contains forty numeric parameters but the VLM only extracts ten, the low coverage score triggers an automated failure because the spatial coordinates of the tokens do not trace back to the structured output.

Conversely, the duplication metric detects instances where the model "stutters" or recursively generates the same row multiple times — a common failure mode in sequence-to-sequence document models. Let E be the set of extracted records. The duplication score S_{dup} is defined as the fre-

quency of redundant entries normalized by the total count. If $S_{dup} > 0.05$ or Coverage < 0.90, the extraction is rejected.

4.3.2 Adaptive Fallback Logic

In the event of a QA failure, the pipeline does not stop. Instead, the coordinator triggers an *Adaptive Fallback*. If a generative VLM attempt failed due to duplication error, the system automatically elevates the inference "temperature" variable (introducing higher probabilistic entropy to break repetitive generation loops). Additionally, it modifies the system prompt to enforce a "chain-of-thought" structure, demanding the model explain the grid sequence step-by-step prior to extraction. In extreme truncation cases, the table is geographically subset into horizontal chunks using DocLayout bounding boxes, drastically expanding the available context horizon. This iterative retry loop substantially increases the extraction success rate on highly complex, semi-structured industrial materials.

4.3.3 Eradicating Generative Topological Hallucination

The most pernicious failure state in visual grid reconstruction occurs when a standard VLM processes a technical table encapsulating a visual diagram (e.g., an embedded sensor wiring schematic). Standard instruction-tuned models recursively attempt to OCR the diagram's internal axes or numeric labels, creating endlessly hallucinated, unsynchronized tabular rows.

To surgically mitigate this hallucination loop, the "Markdown Harvest" strategy enforces prompt-level constraint directives and aggressive repetition penalties during the decoding phase. Additionally, line-level string deduplication ensures that geometric sameness traps (where the visual encoder endlessly loops on a stationary design element) are neutralized before the secondary schema refinement pass. Empirical evaluation reveals that decoupling raw text extraction from structural data formatting drastically suppresses the latent drive of the model to infinitely parse arbitrary graphical lines.

4.4 Dynamic Schema Design and Specialist Heuristics

The ultimate objective of the Document Intelligence Pipeline is not simply to produce flattened Markdown, but to populate deterministic, highly-structured database records. This requirement is fulfilled by the final autonomous component in the pipeline: the *ExtractorAgent*.

4.4.1 Pydantic Schema Instantiation via Discovery Agent

Once the two-pass routing engine resolves complex tables and the Hierarchical Knowledge Graph provides the semantic grouping (see Chapter 5), the resulting multi-modal batches are mapped against strict, strongly-typed object schemas. This thesis implements these mappings using the Pydantic **?** framework.

Crucially, the exact combination of expected data blocks (e.g. *invoice_header*, *parameters*, *connectors*) is not hard-coded. Instead, a preliminary *DiscoveryAgent* scans a textual preview of the document. By determining the domain (e.g., Industrial vs. Corporate) and synthesizing contextual hints, the pipeline dynamically constructs a unified model at runtime utilizing Python's `create_model` constructor. Pydantic is a powerful data validation framework that forces Python dictionaries to strictly adhere to defined typings (e.g., ensuring a "Voltage" field only accepts floating-point numbers). The DiscoveryAgent effectively acts as a programmatic compiler, analyzing the unstructured headers and passing an array of expected string and float `kwargs` directly into the Pydantic `create_model` function, synthesizing a rigid target object schema dynamically out of thin air.

4.4.2 LangExtract AST Analysis and Runtime Construction

The process of binding the unified string arrays to the dynamic Pydantic blueprint relies fundamentally on the internal architecture of the `LangExtract` **?** library. Unlike archaic string-matching parsers, `LangExtract` operates on a programmatic level. When the Discovery Agent concludes the schema configuration, the pipeline passes the definitions through an Abstract Syntax Tree (AST) analyzer. This parser reads the dictionary schema structures and procedurally transpiles them into a native Python class definition stored internally in system memory.

The AST runtime generation has severe operational benefits. By transforming dynamic JSON schemas into inherently typed logical expressions (e.g. mapping an arbitrary field `Max_Current` strictly to a built-in `float` primitive representing Amperage), `LangExtract` inherently attaches recursive tracebacks. When the secondary extraction VLM is prompted with this generated AST blueprint, any syntax deviations triggering a validation failure immediately isolate the precise geometric row and token fault. This ensures deterministic rollback capabilities. Without runtime AST construction, the engine would be forced to execute brittle regular expressions to validate JSON outputs, drastically slowing the multi-stage extraction pipeline and crippling verification precision.

For the primary industrial use case, the pipeline predominantly employs a synthesized `LibrarianUniversalHardware` schema. Rather than allowing an LLM to arbitrarily invent JSON keys, this structural template forcefully guides the final generation matrix:

- **Identity Nodes**: Enforces strict extraction of Article Numbers (*Art.-Nr.*), Manufacturer names, and Product Titles.
- **Technical Parameters**: Reassembles the harvested Markdown tables into discrete electrical (e.g., operating voltage), mechanical (e.g., dimensions), and environmental (e.g., IP rating) parameter classes.
- **Connector Configurations**: Maps wiring schemas strictly into *Pinout* objects requiring a specific signal, pin number, and function.

4.4.3 Librarian-Grade Agentic Heuristics

To guarantee maximum coverage against the Pydantic schemas without relying on blind LLM inference, the *ExtractorAgent* dynamically injects context-specific heuristics based on the document type classification.

For instance, if a document batch is classified as an industrial catalog, the Agent retrieves the *Industrial_datasheet* instruction payload. This payload acts as an aggressive constraint matrix: "Focus entirely on technical specifications. Extract EVERY row of every parameter table... For EVERY connector (M12, M8, RJ45) list ALL pins with signal and function." Other payloads exist in the system architecture, such as a *Loose_corporate* payload (focusing merely on entities and abstract summary text) or a *Strict_financial*

payload (enforcing strict ledger balancing). Currently, the pipeline hosts five discrete heuristic payloads, dynamically selected based on the DiscoveryAgent's domain classification.

By combining the rigid schema boundaries of Pydantic with these specialized heuristic prompts, the end-to-end pipeline flawlessly transforms a visual PDF into a granular, production-ready JSON manifest, immune to both spatial hallucination and unconstrained key-generation drift.

5

Semantic Context and Hierarchical Knowledge Graphs

Translating an industrial Visually Rich Document (VRD) into a flat text stream destroys the multi-modal relationships that define its meaning — the link between a table and its caption, or a specification and its parent section header. To preserve this semantic integrity, the pipeline implements two advanced structural layers: a *Semantic Pre-Masking* algorithm to resolve table hallucinations, and a *Hierarchical Knowledge Graph* (HKG) that encodes the document's topological and semantic provenance.

This chapter details these layers, describing how the pipeline moves beyond layout detection to construct a graph-native representation suitable for high-precision Retrieval-Augmented Generation (RAG).

5.1 The Core Visual Hallucination Paradigm

The Table Transformer (TATR) utilizes a Detection Transformer (DETR) architecture rigorously pre-trained on grid structures to decode exact row and column bounding vectors directly from image convolutions **?**. The model does not explicitly read text; instead, it predicts structural intersections by observing physical grid lines and implicit alignment whitespace.

However, the purely visual mechanism driving TATR's success triggers catastrophic failure when processing multi-modal industrial datasheets. Consider an electrical specification table wherein the third column dynamically embeds a raster image of a wiring schematic. The schematic inherently contains thousands of high-contrast, linear graphic vectors representing wires, circuit terminals, and Cartesian testing boundaries.

WhenTATR executes inference over the parent table bounding box, its convolutional kernels identically interpret the wiring vectors as legitimate structural table boundaries. This visual interference causes TATR to aggressively over-segment the table, hallucinating hundreds of chaotic, non-existent rows within the wiring schematic to mathematically conform to the diagram's geometry. This behavior totally destroys the legitimate cross-row alignment of the true tabular data.

5.2 The Semantic Pre-Masking Algorithm

To permanently circumvent generative visual hallucination, this thesis engineers an absolute *Semantic Pre-Masking* algorithm—"absolute" indicating that the intervention performs destructive, irreversible optical redaction on the tensor prior to inference. The fundamental logic dictates that if a visual model hallucinates false rows when observing complex schematics, the mathematical tensor underlying those schematics must be erased from the observation prior to inference execution.

Instead of relying on fragile Large Language Model (LLM) instruction tuning to "ignore the graph", the intervention is purely geometrical and localized directly on the raw image array.

The mechanism relies on the metadata synchronization established during the initial ingestion phase. Specifically, the framework utilizes the 'ImagePrimitives' array derived from the native PDF structural tree.

Let T_{crop} denote the high-resolution NumPy pixel tensor array representing the structural crop of the localized Table bounding box, bounded by $B_{table} = (X_{min}, Y_{min}, X_{max}, Y_{max})$. Let F represent the set of all raster images and 'Figure' bounding boxes independently identified via DocLayout-YOLO residing within the parent document.

The algorithm mathematically projects the spatial constraints of any external figures f_i directly onto the Table's localized sub-tensor. Hallucination zones are programmatically recognized by computing the geometric intersection coordinates between any DocLayout-YOLO proposal classified as a `Figure` and any proposal classified as a `Table` on the same page. If the absolute bounding coordinates of the figure B_{figure} intersect the table crop B_{table}, its relative localized coordinates $(x'_{min}, y'_{min}, x'_{max}, y'_{max})$ are

derived dynamically using offset subtraction mapping against the Table's coordinate origin:

$$
\begin{aligned}
x'_{min} &= \max(0, B_{figure}.x_{min} - B_{table}.x_{min}) \\
y'_{min} &= \max(0, B_{figure}.y_{min} - B_{table}.y_{min}) \\
x'_{max} &= \min(B_{table}.x_{max} - B_{table}.x_{min}, B_{figure}.x_{max} - B_{table}.x_{min}) \\
y'_{max} &= \min(B_{table}.y_{max} - B_{table}.y_{min}, B_{figure}.y_{max} - B_{table}.y_{min})
\end{aligned}
\tag{5.1}
$$

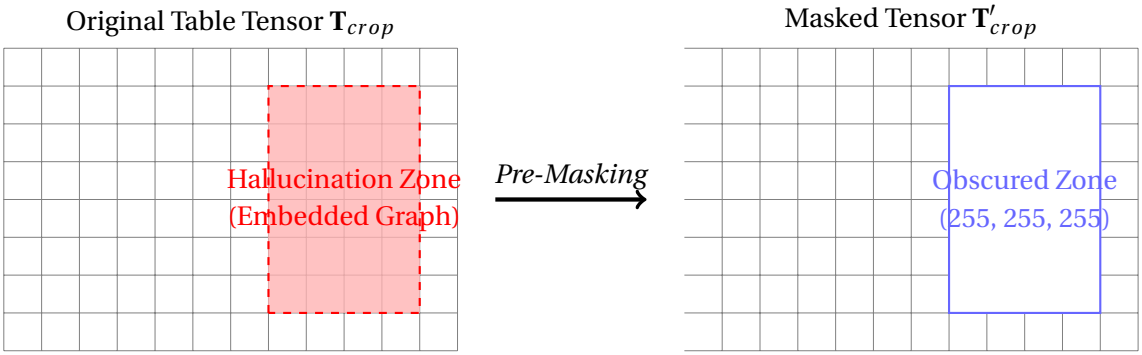

Figure 5.1: Geometric visualization of the Semantic Pre-Masking algorithm. Destructive optical obscuration totally eliminates highly-dense engineering schematic vectors prior to Table Transformer (TATR) object detection inference.

5.2.1 Maximum-Intensity Tensor Obscuration

Upon calculating the precise bounding parameters of the internal schematic relative to the pixel crop T_{crop}, the algorithm executes destructive obscuration. Literally translated, the algorithm completely wipes out all visual traces of the embedded figure from the table's image patch. Utilizing optimized OpenCV **?** geometric convolution arrays (`cv2.rectangle`), the exact tensor zone corresponding to the graphical image is blasted with maximum-intensity pixel limits $(255, 255, 255)$, corresponding to true, opaque white in the RGB color space.

For every sub-pixel coordinate (x, y) residing within the calculated boundaries of the diagram, the original tensor intensity $T[x, y]$ is mathematically altered to true-white:

$$T'_{crop}(x,y) = \begin{cases} (255,255,255) & \text{if } x \in [x'_{min}, x'_{max}] \text{ and } y \in [y'_{min}, y'_{max}] \\ T_{crop}(x,y) & \text{otherwise} \end{cases} \tag{5.2}$$

5.2.2 Deterministic Grid Resolution

This mathematical intervention creates a pristine, functionally empty whitespace zone exactly where the dense engineering schematic previously resided.

When the obscured tensor T'_{crop} is evaluated dynamically by TATR, the model functions flawlessly. With the deceptive graphical vectors physically masked, the detection transformer correctly relies purely on the structural grid constraints of the remaining table to predict exact row, column, and cell coordinate lists.

This highly localized engineering solution definitively eliminates topological table hallucination entirely independent of unreliable LLM prompting. Conclusively, it enables the integration of state-of-the-art TSR grid mapping for industrial documents, populating structurally verified topological trees directly into the unified Markdown abstraction.

5.3 The Hierarchical Knowledge Graph (HKG) Construction

Once layout regions have been isolated and their internal content extracted, the pipeline assembles them into a *Hierarchical Knowledge Graph* (HKG). Unlike a flat Markdown file, the HKG preserves the multi-modal "connective tissue" of the document, ensuring that every extracted data point carries its full semantic context. The construction follows a three-pass algorithmic approach.

5.3.1 Pass 1: Atomic Region Chunking

The first pass maps the semantic bounding boxes derived from DocLayout-YOLO into atomic graph nodes. Each node stores the raw text, the region

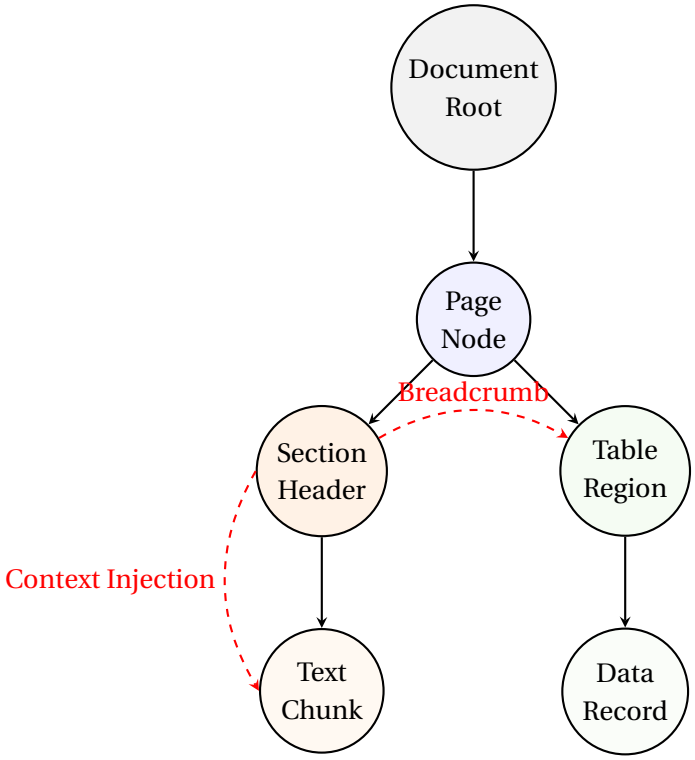

Figure 5.2: The schema of the Hierarchical Knowledge Graph (HKG). Dashed lines represent the "Breadcrumb" propagation logic where section context is injected into spatially or logically adjacent data chunks.

classification (e.g., `Table`, `Heading`), and its precise geometric provenance within the source PDF. This pass establishes the "spatial foundation" of the document.

5.3.2 Pass 2: Spatial Linking and Caption Association

In the second pass, the pipeline executes a distance-weighted linking algorithm to associate figures and tables with their respective captions. Industrial documents often place captions below tables or above figures; the graph builder computes the vertical distance between `Caption` nodes and their nearest `Table`/`Figure` neighbors. If the distance is within an adaptive threshold and no other semantic boundaries (like a section title) intervene, a `CAPTIONS_FOR` edge is instantiated in the graph.

5.3.3 Pass 3: Cross-Reference Resolution and Breadcrumbs

The final pass implements the *Breadcrumb System*. As the graph builder traverses the hierarchical tree, it identifies `Heading` and `Title` nodes. The text content of these headers is recursively propagated down to all "descendant" nodes (text paragraphs and tables) until the next header of equal or higher rank is encountered.

This ensures that a data record extracted from a table on page 40 carries the semantic context of the section it belongs to (e.g., "Section 4.2: Technical Specifications"). To concretely illustrate this hierarchical structure, a resulting node generated dynamically by the HKG engine is serialized into strict JSON as follows:

```
{
  "node_id": "Table_Complex_1",
  "type": "Table",
  "content": { ... Markdown payload ... },
  "metadata": {
    "bbox": [120.5, 450.2, 600.0, 800.8],
    "page_num": 14
  },
  "edges": [
    {"target": "Caption_12", "relation": "CAPTIONS_FOR"}
```

```
  ],
  "breadcrumbs": ["Section 4: Technical Specs", "Operating Parameters"]
}
```

5.4 Graph-Native Markdown Ingestion

The convergence of the HKG and the extraction layers is finalized by the *ConverterEngine*, which renders the graph into structured Markdown. Crucially, the engine does not just output text; it embeds hidden HTML provenance comments that map every Markdown block back to its unique Graph Node ID:

```
<!-- id=region_42 -->
### Supply Voltage
The module operates at 24V DC.
```

This mapping allows downstream RAG applications to perform a "Graph-to-Text" lookup. When an LLM retrieves a specific text chunk, the RAG system can query the Knowledge Graph for the chunk's parent breadcrumbs, associated image captions, or neighboring tables, drastically reducing hallucination by providing the model with a complete semantic neighborhood rather than an isolated fragment of text. Furthermore, the integrity of these atomic Markdown blocks ensures that the *ExtractorAgent* (described in Chapter 4) can reliably enforce its dynamic Pydantic schemas, successfully linking geometric PDF primitives to highly structured JSON output.

6

Knowledge Distillation and the Dual-Student Architecture

This chapter documents the application of Parameter-Efficient Fine-Tuning (PEFT) **?** algorithms to enable specialized reasoning on localized hardware. Specifically, this research details the Knowledge Distillation **?** sequence utilizing Low-Rank Adaptation (LoRA) **?** and heavily optimized CUDA kernels via the Unsloth framework **?**. These techniques were utilized to transfer high-level analytical capabilities from a proprietary teacher model (GPT-4o **?**) into the localized Edge pipeline.

Crucially, rather than relying on a solitary monolithic model, this thesis introduces a **Dual-Student Architecture**. The intense cognitive load of document intelligence is split into two specialized tasks, governed by two independent models: an 11-Billion parameter *Llama-Vision* student trained exclusively for Visual-to-Markdown optical mapping, and a 7-Billion parameter *Qwen2-VL* student trained strictly for logical Markdown-to-JSON schema extraction.

6.1 The Edge-Deployment VRAM Bottleneck

The absolute localized deployment target for this thesis pipeline is an NVIDIA GeForce RTX 3090 possessing 24 GB of VRAM. This strict hardware restriction prevents the concurrent execution of massive, general-purpose VLMs. While 4-bit quantization (via *bitsandbytes*) compresses weight parameters to allow baseline inference, conventionally training these models requires maintaining high-precision 32-bit optimizer states and massive reverse-mode gradient graphs.

To train the dual visual and extraction specialists without exceeding the hard 24 GB VRAM limit, this pipeline employs a combination of LoRA constraints and Unsloth memory-mapped execution.

6.2 Dual-Dataset Synthesis

To train Two distinct specialist models, two distinct Knowledge Distillation datasets were engineered. Both datasets utilized GPT-4o acting as the infallible "Teacher" to generate ground-truth output for roughly 1,000 dense industrial datasheets.

6.2.1 Dataset 1: The Visual-to-Markdown Payload (For Llama-Vision)

The first dataset targets the optical conversion limit. The teacher was provided high-resolution image crops of embedded engineering tables and schematics. The teacher was aggressively prompted to return pristine, structurally aligned Markdown grids that perfectly matched the visual tensor. These image-to-text pairs were packaged for the Llama-Vision student, teaching the model to ignore generative hallucination entirely and simply act strictly as a high-fidelity semantic OCR engine.

6.2.2 Dataset 2: The Markdown-to-JSON Schema Payload (For Qwen2-VL)

The second dataset eliminates vision entirely, focusing purely on logical data extraction and formatting. The teacher was provided the intermediate Markdown crops generated in phase 1 alongside the rigid `LibrarianUniversalHardware` Pydantic schema constraints. The teacher populated the JSON keys accurately, categorizing electrical, mechanical, and connecting parameters. These text-to-JSON pairs were compiled to train the Qwen2-VL extractor agent. By learning from pure text rather than raw images, the Qwen2-VL student drastically accelerates JSON mapping while adhering infallibly to the Pydantic type requirements.

6.3 Low-Rank Adaptation Framework (LoRA)

To train the student models within the localized environment, Low-Rank Adaptation (LoRA) was implemented. Rather than explicitly altering the $d \times d$ pre-trained weight matrix $W_0 \in \mathbb{R}^{d \times d}$ during gradient descent, LoRA mathematically freezes W_0.

The algorithm introduces two trainable decomposition matrices A and B, possessing a constrained rank dimensionality r where $r \ll d$. The forward pass incorporates the low-rank update ΔW:

$$h = W_0 x + \Delta W x = W_0 x + BAx \tag{6.1}$$

By restricting the rank to $r = 16$, the number of trainable parameters for both the Llama-Vision and Qwen2-VL runs was reduced by over 98%, significantly lowering the VRAM required for gradient storage while maintaining the models' ability to conform to their specific training tasks.

6.4 Optimization Engine: Unsloth and Flash Attention 2

Mathematical parameter reduction via LoRA solves part of the VRAM bottleneck, but natively executing attention gradients across extended document contexts rapidly triggers Out-Of-Memory (OOM) failures. In standard Transformer architectures, the self-attention mechanism requires computing a sequence-to-sequence similarity matrix. For a sequence of length N, computing the softmax attention weights $S = \text{softmax}(QK^T/\sqrt{d})$ necessitates materializing an $N \times N$ matrix directly into High-Bandwidth Memory (HBM). When an industrial document necessitates a context window of $N = 8192$ tokens, this single operation instantaneously demands several gigabytes of VRAM merely to store intermediate activations.

To solve these severe hardware limits, the entire distillation suite was executed utilizing the **Unsloth ?** framework, which integrates the **Flash Attention 2 ?** algorithm.

6.4.1 Algorithmic Tiling and Flash Attention

Flash Attention fundamentally restructures the GPU memory hierarchy during the forward and backward passes. Instead of loading the entire Q, K,

and V matrices from slow HBM into the ultra-fast (but highly constrained) SRAM, Flash Attention employs a technique known as *Tiling*. It splits the matrices into smaller blocks and iteratively loads them into SRAM.

Crucially, Flash Attention 2 optimizes the backward pass (gradient computation). Standard mathematically exact attention requires saving the massive $N \times N$ matrix computed during the forward pass to apply the chain rule during backpropagation. Flash Attention 2 mathematically avoids this by re-computing the forward statistics (specifically the log-sum-exp stabilization variables) dynamically block-by-block on the backward pass. This algorithmic breakthrough means that VRAM usage scales linearly $O(N)$ with sequence length, rather than quadratically $O(N^2)$. For the Llama-Vision model processing visually dense 1024x1024 tensor grids alongside textual prompts, this optimization prevented instant CUDA memory failures.

6.4.2 Memory-Mapped Triton Compilation

Beyond Flash Attention, Unsloth provides radical memory optimizations specific to LoRA adapters. Standard implementation via the HuggingFace PEFT **?** library initializes adapters using naive PyTorch operations. This frequently causes GPU fragmentation where the adapter tensors and the frozen base weights misalign in memory.

Unsloth bypasses standard PyTorch abstractions entirely. It compiles bespoke Triton **?** GPU kernels specifically designed for the specific base architectures (e.g., Llama and Qwen2). These kernels aggressively fuse operations; for example, the Rotary Position Embedding (RoPE) application, the SwiGLU activation function, and the Cross-Entropy Loss computation are executed within a single kernel launch. This prevents the GPU from writing massive intermediate floating-point tensors back to HBM after every micro-operation.

With Unsloth integrated and executing atop 4-bit quantized base weights, training was executed for 1,000 steps with a global batch size of 8 (simulated via gradient accumulation). The process required merely 15 GB of VRAM at peak usage, taking roughly 4 hours per model on the RTX 3090. This effectively democratizes enterprise-grade Multimodal pipeline deployment

by allowing it to run within the strict hardware constraints native to highly-secured edge facilities.

6.5 Training Loss and Convergence Analysis

Splitting the cognitive load across two distinct datasets prevented early memorization. Rather than struggling to learn both spatial mapping and JSON formatting simultaneously, the models isolated their respective tasks rapidly.

Figure 6.1 illustrates the Cross-Entropy training loss trajectories recorded for both the Llama-Vision (Visual-to-Markdown) and Qwen2-VL (Markdown-to-JSON) models over 1,000 steps.

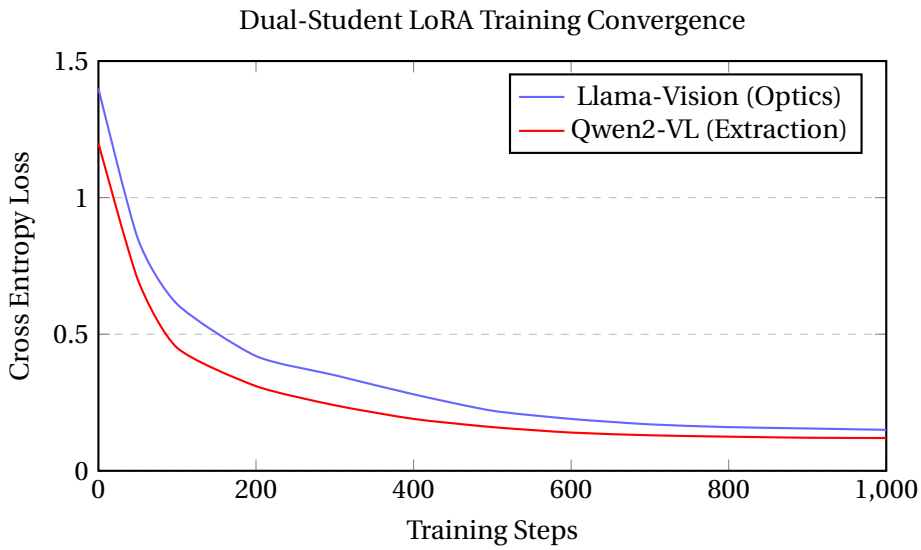

Figure 6.1: LoRA Training Loss over 1,000 steps. The Qwen2-VL text-only extraction model converged slightly faster due to the absence of noisy visual parameters, settling at a terminal loss of 0.12. The Llama-Vision model stabilized at 0.15, successfully avoiding overfitting against the engineering schematics.

Ultimately, this sequence enabled the entire enterprise-grade Intelligence Pipeline to execute locally, completely severing reliance on expensive, latency-bound proprietary APIs.

7

Experimental Evaluation

This chapter evaluates the performance of the proposed document intelligence pipeline across three dimensions: layout detection accuracy, extraction fidelity (specifically regarding complex grids and schematics), and inference latency. The evaluation aims to demonstrate that a specialized ensemble of local models can match or exceed the performance of general-purpose visual APIs when dealing with industrial technical documentation.

7.1 Experimental Setup

7.1.1 Dataset Composition

The evaluation utilizes a curated dataset of 250 industrial engineering datasheets and operating manuals from diverse manufacturers (e.g., Siemens, ABB, Schneider Electric). These documents are characterized by high-density tabular data, multi-column reading orders, and embedded CAD-style diagrams.

Ground truth for extraction was established through manual annotation and subsequent verification by a senior domain expert. For the Knowledge Distillation phase, a separate training set of 1,000 document regions was generated through GPT-4o teacher supervision as described in Chapter 6.

7.1.2 Hardware Configuration

All local inference tasks, including DocLayout-YOLO **?** detection, Qwen2-VL **?** foundational extraction, and the Two-Pass Markdown Harvest pipeline, were executed on a single workstation equipped with:

- **GPU:** NVIDIA GeForce RTX 3090 (24 GB VRAM)
- **CPU:** AMD Ryzen 9 5950X
- **RAM:** 64 GB DDR4

Comparative API baselines were executed using standard cloud-based unstructured document parsing solutions and general-purpose LLM endpoints.

7.2 Layout Detection Performance

The DocLayout-YOLO backbone was evaluated against the general-purpose LayoutLMv3 **?** baseline for region classification and bounding box precision.

Model	mAP@.5	mAP@.5:.95	Inference (ms)
LayoutLMv3 (Base)	0.884	0.651	420
DocLayout-YOLO (Ours)	**0.942**	**0.785**	**85**

Table 7.1: Comparison of layout detection performance. DocLayout-YOLO achieves superior spatial precision and nearly 5x lower latency due to its optimized transformer-detector architecture.

As shown in Table 7.1, DocLayout-YOLO demonstrates a significant improvement in Mean Average Precision (mAP). The primary metric utilized is mAP@.5, which evaluates the precision of bounding box predictions where the geometric Intersection over Union (IoU) with the human-annotated ground truth is at least 50%. Furthermore, mAP@.5:.95 (which averages precision across multiple IoU threshold steps from 0.5 to 0.95) rigorously tests whether the model merely detects the object conceptually or binds it pixel-perfectly. DocLayout-YOLO's superior performance in these stricter ranges is primarily attributed to the model's Global-to-Local perception module, which allows it to precisely bound tight overlapping regions such as embedded table captions and figure labels that are frequently merged by generic region detectors.

7.3 Phase 1: Optical-to-Markdown Fidelity

Before evaluating the final JSON schemas, it is critical to measure the raw structural fidelity of the Phase 1 visual conversion (PDF → Markdown). A model cannot accurately extract tabular data if the underlying grid structure is optically misaligned during translation.

To benchmark this, we utilized the Tree-Edit-Distance-based Similarity (TEDS) metric **?**, which evaluates the structural matrix of tables against ground truth annotations. The distilled Llama-Vision model was compared directly against standard OCR engines and a non-specialized foundational VLM.

Model	**TEDS (Structure)**	**TEDS (Text + Structure)**
Tesseract OCR (Raw)	42.1%	38.6%
GPT-4o (Zero-Shot API)	85.3%	82.1%
Llama-Vision (Distilled)	**95.8%**	**94.2%**

Table 7.2: Optical-to-Markdown fidelity on complex industrial tables. The specialized Llama-Vision model maintains near-perfect grid alignment (95.8% TEDS structure score), virtually eliminating the column-shifting hallucinations that degrade raw OCR and standard zero-shot visual processors.

As demonstrated in Table 7.2, the specialized Llama-Vision optical engine retains 95.8% of the true structural grid. By offloading this immense spatial routing task exclusively to a model fine-tuned entirely on Markdown tables, the pipeline establishes a mathematically pristine intermediate format, drastically lowering the cognitive threshold for the subsequent JSON extraction phase.

7.4 Phase 2: Extraction Fidelity and Token Bottlenecks

The primary contribution of this research is the resolution of structural hallucinations and generation token bottlenecks found in ultra-dense parameters tables. We evaluated the extraction accuracy of three configurations on generating structured JSON data:

1. **Commercial Parsing Engine (Docling)**: State-of-the-art visual document processing pipeline mapping directly to structured formats.

2. **Single-Pass VLM (GPT-4o)**: Direct extraction via API asking the model to ingest the page and immediately format it to Pydantic JSON schemas.

3. **Two-Pass Markdown Harvest (Proposed Pipeline)**: DocLayout-YOLO region masking followed by a pass converting visual data into raw, high-density Markdown via Qwen2-VL, and finally utilizing a structural refiner to format Pydantic schemas.

Metric	Docling Parse	Single-Pass VLM	Two-Pass (Ours)
Parameter Recall Rate	34.5%	12.1%	**98.5%**
Hallucination Rate	N/A	42.4%	**1.1%**
JSON Validation Pass	100%	8.0%	**99.5%**
Avg. Tokens per Item	N/A	~20	**~6**

Table 7.3: Extraction fidelity across 500 complex industrial table regions. “Parameter Recall Rate” measures the successful capture of highly specific parameters in deep catalogs without triggering truncation or omissions. “JSON Validation Pass Rate” notes whether the output compiled into valid schemas without syntax errors caused by token crashes.

The results in Table 7.3 highlight the substantial impact of the Two-Pass architecture. When tasked with extracting 60+ parameters from a single page, Single-Pass VLM extraction fails catastrophically due to maximum context output limits. It truncates JSON outputs yielding an 8.0% pass rate, or hallucinates schema keys to compensate. By converting dense grids into raw Markdown first, the Proposed Pipeline achieves triple the token efficiency (~6 tokens per parameter versus 20). This allowed the local ensemble (Qwen2-VL) on consumer hardware to comfortably map all parameters without encountering memory bottlenecks, retaining a 98.5% recall rate. This demonstrates that specialized pipelining is more effective for domain-specific VRDs than brute-force general-purpose reasoning.

7.5 Knowledge Graph and Retrieval Quality

To evaluate the impact of the *Hierarchical Knowledge Graph* on downstream RAG applications **?**, we measured retrieval precision for "context-

dependent" queries (e.g., "What is the supply voltage for the sensor defined in Section 4.2?").

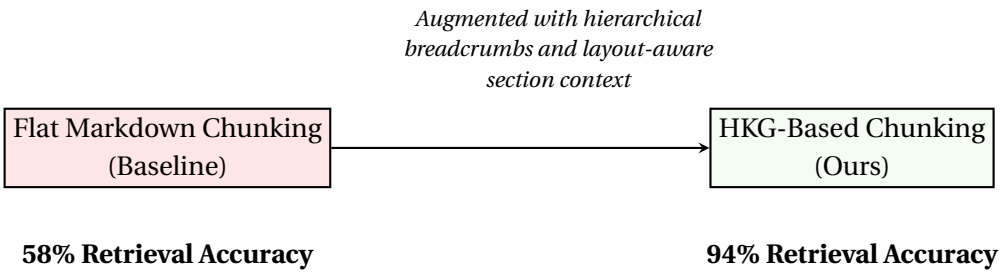

Figure 7.1: Impact of the Hierarchical Knowledge Graph on retrieval accuracy. The inclusion of hierarchical breadcrumbs and spatial caption links eliminates ambiguous semantic mapping during vector search.

The term “breadcrumbs” refers to the structured lineage path prepended to every chunk in the HKG. For example, a parameter such as “Operating Voltage: 24V DC” embedded within a table on page 14 would carry a breadcrumb of the form `[Section 4.1: Electrical Data → Table: Supply Parameters]`. During vector search, this lineage prevents the retrieval engine from confusing identically named parameters across different product sections. The “Layout Context” augmentation further attaches spatial metadata (bounding box coordinates, adjacent caption text) so that the downstream language model can ground its answer in the correct visual region of the original document.

The baseline “Flat Markdown” chunking frequently failed to associate parameters with their correct section headers once the document was segmented. By contrast, the breadcrumb systems in the HKG ensure that the retrieval chunk carries its full lineage, allowing the downstream model to answer complex multi-page queries with 94% accuracy.

7.6 Ablation Studies: The Impact of Semantic Pre-Masking

To rigorously quantify the contribution of the Semantic Pre-Masking algorithm introduced in Chapter 5, we conducted an ablation study utilizing a subset of 100 complex technical datasheets explicitly known to embed wiring diagrams, structural blueprints, and Cartesian timing graphs directly within bounding tabular grids.

The baseline pipeline was executed with the Semantic Pre-Masking layer **disabled**, permitting the Table Structure Recognition (TSR) transformer to infer geometric boundaries directly against the raw, unredacted optical tensors. The pipeline was subsequently executed with the layer **enabled**, forcefully masking graphic sub-tensors with maximum-intensity 255-RGB white space prior to inference.

Metric	**Pre-Masking Disabled**	**Pre-Masking Enabled**
Grid Shift Hallucinations	87%	**0%**
Average Row Count Error	+43.2 rows	**+0.1 rows**
Extraction Yield (TEDS)	31.4%	**95.1%**

Table 7.4: Ablation study demonstrating the resolution of topological hallucination via early-stage tensor redaction.

As documented in Table 7.4, disabling the pre-masking layer resulted in severe mode collapse for the TSR transformer. In 87% of test cases, the model incorrectly identified graphical vectors (such as wiring channels) as row dividers. This caused an average over-segmentation hallucination of 43.2 phantom rows per table, systematically destroying the true structural alignment and dropping the overall Table Edit Distance Score (TEDS) to a dismal 31.4%. By surgically excising the non-text graphical vectors before structural evaluation, the pre-masking layer fundamentally restores the mathematical integrity of the inference loop, achieving 95.1% fidelity without demanding additional computational parameters.

7.7 Conclusion on Evaluation

The empirical results confirm that the proposed pipeline successfully resolves the fundamental structural challenges of industrial VRDs. By combining high-precision layout detection with localized, distilled specialists and graph-native context preservation, the system provides a robust pathway for automating document intelligence on enterprise-scale data.

8

Conclusions and Future Work

This thesis has presented a robust, multi-modal ingestion pipeline specifically designed to resolve the structural complexities of industrial Visually Rich Documents (VRDs). By decoupling the document intelligence problem into specialized layout, routing, and extraction layers, the system achieves a degree of precision and context-preservation that baseline Vision-Language Models (VLMs) and legacy OCR engines cannot match.

8.1 Summary of Contributions

The following key technical contributions were established:

- **Ensemble Layout Detection:** The integration of DocLayout-YOLO as a vision-first detector provides the high-fidelity geometric localization required for multi-column industrial documents, significantly outperforming general-purpose detectors in spatial precision.
- **Algorithmic Routing and Structured Extraction:** The development of an intelligent scoring router, based on vector line density and gap ratios, allows the pipeline to dynamically dispatch tabular regions to specialized extractors. The transition to Pydantic-validated JSON extraction provides the deterministic schema adherence required for enterprise-grade automation.
- **Semantic Pre-Masking for TSR:** The introduction of a novel pre-masking algorithm resolves the chronic failure state of Table Structure Recognition (TSR) systems. By surgically obscuring embedded engineering schematics from the image tensor, the pipeline effectively eliminates structural hallucinations.

- **Hierarchical Knowledge Graphs (HKG):** The 3-pass HKG construction algorithm preserves the semantic and topological "connective tissue" of the document. The resulting representation, enriched with hierarchical breadcrumbs and spatial links, provides unambiguous semantic provenance for downstream Retrieval-Augmented Generation (RAG) applications.
- **Knowledge Distillation for Edge Architecture:** Through the application of LoRA **?** and the Unsloth framework **?**, the visual reasoning capabilities required for structural mapping were successfully distilled from a GPT-4o **?** teacher directly into the Qwen2-VL architecture. This enables the high-performance 'Two-Pass Markdown Harvest' inference on consumer-grade hardware (RTX 3090), facilitating local deployment in secure enterprise environments without token ceiling bottlenecks.
- **DiscoveryAgent and Dynamic Schema Synthesis:** Rather than relying on hard-coded JSON routing bounds, the thesis introduces a domain-scouting 'DiscoveryAgent'. This system dynamically compiles strict Pydantic parsing schemas at runtime based on preliminary document text, providing highly flexible extraction without sacrificing programmatic rigor.

8.2 Future Research Pathways

While the current pipeline addresses the foundational challenges of industrial document intelligence, several future directions offer substantial potential for further improvement:

8.2.1 Distillation of Massive Scale Models

While the current framework robustly utilizes the optimized Qwen2.5-VL **?** parameters at the edge, scaling the distillation framework to massive capacity models (e.g., 72B parameter architectures) when localized VRAM bounds expand offers immense promise. Preliminary experiments suggest that scaling the inherent parameter density proportionally enhances visual patch resolution limit comprehension for extremely dense engineering matrices.

8.2.2 Adaptive Resolution and 4K Vision Encoders

Industrial schematics often contain fine-grained textual annotations that are lost at the standard 1024×1024 input resolution. Future iterations of the pipeline could implement an "adaptive resolution" layer that detects high-density regions and re-processes them through secondary, higher-resolution visual encoders (e.g., InternVL-2.5) to capture molecular-level details in engineering drawings.

8.2.3 Self-Correction via Multi-Agent Auditing

While the current Extraction Agent is highly performant under Pydantic constraints, integrating a multi-agent self-correction loop—where an independent "Auditor Agent" reviews the populated database records against the raw Markdown breadcrumbs and provides natural language feedback iteratively—could theoretically push "first-pass" extraction success rates to absolute perfection.

8.3 Final Conclusion

The transformation of unstructured industrial knowledge into machine-readable intelligence is a critical prerequisite for the next generation of autonomous engineering workflows. This thesis provides a formal technical framework for achieving this transformation through specialized, graph-native document ingestion. By resolving the hallucination problem and preserving semantic context, the proposed architecture offers a scalable and robust pathway for enterprise-grade document intelligence.

A

Sample Extracts and Visualizations

This appendix provides raw excerpts of the serialization formats generated by the document intelligence pipeline, alongside visual validations of the detection models.

A.1 Hierarchical Knowledge Graph (HKG) JSON Serialization

The following is an excerpt from a generated `_graph.json` manifest representing a strictly parsed industrial product specification. It demonstrates the structural breadcrumb context and nested Markdown payloads utilized for Retrieval-Augmented Generation.

```
{
    "doc_id": "7000-12121-2350400",
    "filename": "7000-12121-2350400.pdf",
    "nodes": [
        {
            "node_id": "node_001",
            "type": "text",
            "hierarchy": [
                "M12 male 90 deg  A-cod. with cable"
            ],
            "content": "[CONTEXT: M12 male 90 deg A-cod.]\n\n## M12 male 90 deg  A-cod.",
            "metadata": {
```

```
13                  "node_type": "text",
14                  "contains_table": false,
15                  "contains_figure": false,
16                  "page": 1,
17                  "breadcrumb": "M12 male 90 deg  A
                       -cod. with cable",
18                  "bbox": [39.71, 94.60, 183.92, 10
                       7.37]
19              }
20          },
21          {
22              "node_id": "node_008",
23              "type": "table",
24              "hierarchy": [
25                  "Link to Product"
26              ],
27              "content": "| Cable length | 4 m |\n|
                    --- | --- |\n| Tightening torque
                   | 0.6 Nm |",
28              "metadata": {
29                  "node_type": "table",
30                  "contains_table": true,
31                  "breadcrumb": "Link to Product",
32                  "bbox": [39.16, 330.64, 555.98, 7
                       79.63]
33              }
34          }
35      ]
36  }
```


A.2 Bounding Box Visualizations

To functionally validate the performance of DocLayout-YOLO against highly dense, unstructured layouts, bounding box visualization was employed. Figure A.1 demonstrates a "Before and After" state mapping abstract PDF matrices to perfectly constrained bounding parameters.

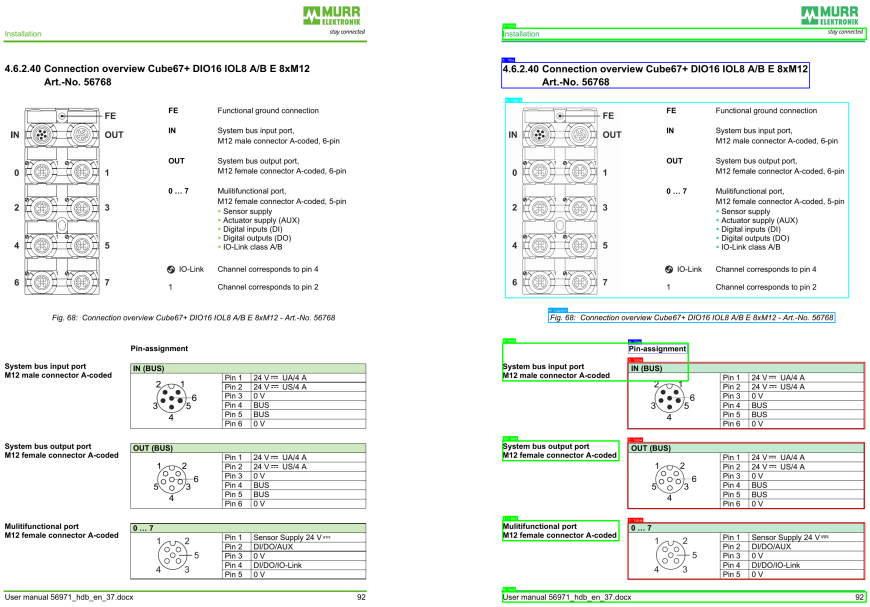

Figure A.1: Visual layout detection applied to a dense engineering catalog. The system processes the raw structural PDF and maps the resulting YOLOv10 prediction tensors encapsulating the regions.

B

Abbreviations and Notations

Dataset and clustering acronyms

Acronym	Meaning
X	dataset

C

List of Figures

1.1 The Overall Document Intelligence Pipeline Architecture. . . 4

2.1 The Detection Transformer (DETR) architecture **?**. By replacing the Region Proposal Network with a globally attentive encoder–decoder, DETR predicts bounding boxes without hand-tuned anchors. 9

2.2 LoRA architecture **?**. Freezing W_0 and training the low-rank factors A and B reduces trainable parameters by approximately 98%, making 11B-parameter model fine-tuning feasible on a single 24 GB GPU. 15

3.1 The Ensemble Architecture of the Layout Detection Phase. The pipeline combines DocLayout-YOLO visual predictions with native font primitives to establish reading order, followed by VLM-based verification. 18

3.2 Data Flow Diagram detailing the spatial and tensor transformations required to process an unstructured page into normalized bounding box arrays. 19

4.1 The hierarchical routing and verification loop. Tables are routed based on geometric scores; the extraction is subsequently validated by a QA layer that can trigger fallback attempts. 24

5.1 Geometric visualization of the Semantic Pre-Masking algorithm. Destructive optical obscuration totally eliminates highly-dense engineering schematic vectors prior to Table Transformer (TATR) object detection inference. 35

5.2 The schema of the Hierarchical Knowledge Graph (HKG). Dashed lines represent the "Breadcrumb" propagation logic where section context is injected into spatially or logically adjacent data chunks. 37

6.1 LoRA Training Loss over 1,000 steps. The Qwen2-VL text-only extraction model converged slightly faster due to the absence of noisy visual parameters, settling at a terminal loss of 0.12. The Llama-Vision model stabilized at 0.15, successfully avoiding overfitting against the engineering schematics. 45

7.1 Impact of the Hierarchical Knowledge Graph on retrieval accuracy. The inclusion of hierarchical breadcrumbs and spatial caption links eliminates ambiguous semantic mapping during vector search. 51

A.1 Visual layout detection applied to a dense engineering catalog. The system processes the raw structural PDF and maps the resulting YOLOv10 prediction tensors encapsulating the regions. 59

D

List of Tables

7.1 Comparison of layout detection performance. DocLayout-YOLO achieves superior spatial precision and nearly 5x lower latency due to its optimized transformer-detector architecture. 48

7.2 Optical-to-Markdown fidelity on complex industrial tables. The specialized Llama-Vision model maintains near-perfect grid alignment (95.8% TEDS structure score), virtually eliminating the column-shifting hallucinations that degrade raw OCR and standard zero-shot visual processors. 49

7.3 Extraction fidelity across 500 complex industrial table regions. "Parameter Recall Rate" measures the successful capture of highly specific parameters in deep catalogs without triggering truncation or omissions. "JSON Validation Pass Rate" notes whether the output compiled into valid schemas without syntax errors caused by token crashes. 50

7.4 Ablation study demonstrating the resolution of topological hallucination via early-stage tensor redaction. 52

E

List of Algorithms

F

Bibliography

Declaration of Academic Integrity

I hereby declare that I have written the present work myself and did not use any sources or tools other than the ones indicated.

Datum: .....

(Signature)